

Загадан массив из N битов – $a_0, \dots, a_{N-1}$. За один запрос можно выбрать подмножество позиций и инвертировать биты на этих позициях. Инвертирование бита 0 изменяет его на 1, а инвертирование бита 1 изменяет его на 0. После каждого запроса вам сообщается длина самого длинного последовательного отрезка из 1 в получившемся массиве. **Изменения, внесенные запросами, постоянны, то есть биты остаются в новом состоянии и следующий запрос применяется к новому массиву.**

Вы хотите найти самый длинный отрезок единиц в массиве **после выполнения всех запросов** (если отрезков равной максимальной длины несколько, можно найти любой из них). Напишите программу, которая будет решать эту задачу, сделав поменьше запросов.

Implementation details

Каждый тест состоит из T подтестов.

Вам необходимо реализовать функцию со следующей сигнатурой:

```
std::pair<int,int> find_longest_subarray_of_ones(int n);
```

Она будет запущена T раз на каждом тесте, на вход она получает число N – длину загаданного массива. Функция должна вернуть пару $\text{pair}\{l, r\}$, где l представляет собой индекс самой левой единицы в найденном отрезке, а r – индекс самой правой единицы в нем.

Для выполнения запросов необходимо использовать функцию *flip_bits*:

```
int flip_bits(const std::vector<bool> &flips);
```

Она получает вектор из N битов $\text{flips}_0, \dots, \text{flips}_{n-1}$, где $\text{flips}_i = 1$ означает, что бит a_i следует инвертировать. После инвертирования этих битов функция возвращает длину самого длинного отрезка единиц в получившемся массиве.

Вам следует отправить на проверку файл `ones.cpp`, который содержит функцию *find_longest_subarray_of_one*. Он может также содержать другие вспомогательные функции и код, но не должен содержать функцию *main*. Также он не должен пытаться читать из стандартного ввода или писать в стандартный вывод. Для корректной работы ваша программа должна подключить заголовочный файл `ones.h`, используя инструкцию препроцессора:

```
#include "ones.h"
```

Local testing

Вам доступен файл `Lgrader.cpp`, который может быть скопирован с вашей программой, чтобы тестировать локально. Он считывает со стандартного потока ввода число подтестов T , а затем сами тесты. Для каждого подтеста будет задано число N и затем на следующей строке числа $a_0 \dots a_{N-1}$. Если тест решен успешно, то выводится 1 и затем максимальное количество запросов, которое было сделано. Иначе выводится 0.

Constraints

$$T = 5$$

$$1 \leq N \leq 10^4$$

$$0 \leq a_i \leq 1$$

Scoring

Если хотя бы на одном подтесте получен неверный ответ, баллы за задачу равны 0. Иначе баллы за задачу зависят от максимального количества запросов Q , которое делает ваша программа на одном подтесте. Баллы за задачу равны:

$$\left(\frac{85}{\max(85, Q)}\right)^{0.294} \times 100$$

Sample Communication

Пусть исходный массив имеет вид 1, 0, 1. Один из вариантов взаимодействия может выглядеть так:

Contestant's function	Jury's program
	Calls find_longest_subarray_of_ones(3)
Calls flip_bits({0, 0, 0})	Returns the value 1
Calls flip_bits({0, 1, 0})	Returns the value 3
Returns the value {0, 2}	